

EE 475 Introduction to Image Processing

Homework 01

Boran Aybak Kılıç
Student ID: 2022401207

1 Scaling

Given an image as a matrix A , we need to scale it to the range $[0, 1]$.

1.1 Theoretical Function

To come up with a normalization method that will create a new image scaled to the range $[0, 1]$, we impose the following constraints:

- The darkest pixels (originally $\min(A)$) map to 0
- The brightest pixels (originally $\max(A)$) map to 1
- All intermediate values are linearly interpolated between 0 and 1.
- Relative intensity order and linearity of intensity differences between any 2 pixels must be preserved.

The transformation that achieves these constraints is an affine transformation. This affine transformation performs the following operations:

1. **Translation:** Subtracting $\min(A)$ shifts the minimum value to 0
2. **Scaling:** Dividing by $(\max(A) - \min(A))$ scales the range to $[0, 1]$

Therefore, normalization function is given by:

$$A_{\text{normalized}} = \frac{A - \min(A)}{\max(A) - \min(A)} \quad (1)$$

The resulting image $A_{\text{normalized}}$ will have pixel values in the exact range $[0, 1]$ where the required functions for this normalization process are:

- $\min(A)$ – finds the minimum pixel value in the image matrix
- $\max(A)$ – finds the maximum pixel value in the image matrix

1.2 Implementation

The implementation is provided in MATLAB. The code handles edge cases such as constant images (where all pixels have the same value) to avoid division by zero. The complete code is provided in Appendix A.

2 Interpolation

The provided image files of the chronometer are obtained from the original file `Chronometer.tif` by sub-sampling the original file at each direction.

2.1 Image Resolutions

The resolutions of the provided image files are:

- *Chronometer.tif* has size **2136 x 2140**
- *Chronometerlow2.tif* has size **1068 x 1070**
- *Chronometerlow34.tif* has size **712 x 535**
- *Chronometer5.tif* has size **428 x 428**

2.2 Interpolation Algorithms

2.2.1 Bilinear Interpolation

Motivation: When resizing an image, we need to estimate intensity values at positions that do not correspond to integer pixel coordinates in the original image. Bilinear interpolation provides a computationally efficient method to estimate these values using weighted averages of the four nearest neighboring pixels.

Mathematical Derivation:

Consider a point (x, y) in the input image coordinate system where we wish to interpolate the intensity value. Let the four surrounding pixels be located at integer coordinates:

- (x_0, y_0) – top-left neighbor
- (x_1, y_0) – top-right neighbor
- (x_0, y_1) – bottom-left neighbor
- (x_1, y_1) – bottom-right neighbor

where $x_0 = \lfloor x \rfloor$, $x_1 = x_0 + 1$, $y_0 = \lfloor y \rfloor$, and $y_1 = y_0 + 1$.

The bilinear interpolation proceeds in two stages:

Step 1: Horizontal Interpolation First, we interpolate along the horizontal direction at both y_0 and y_1 :

$$I(x, y_0) = I(x_0, y_0) \cdot (x_1 - x) + I(x_1, y_0) \cdot (x - x_0) \quad (2)$$

$$I(x, y_1) = I(x_0, y_1) \cdot (x_1 - x) + I(x_1, y_1) \cdot (x - x_0) \quad (3)$$

Since $x_1 - x_0 = 1$, we can define the fractional horizontal distance as:

$$\Delta x = x - x_0 \in [0, 1] \quad (4)$$

This simplifies the horizontal interpolations to:

$$I(x, y_0) = I(x_0, y_0) \cdot (1 - \Delta x) + I(x_1, y_0) \cdot \Delta x \quad (5)$$

$$I(x, y_1) = I(x_0, y_1) \cdot (1 - \Delta x) + I(x_1, y_1) \cdot \Delta x \quad (6)$$

Step 2: Vertical Interpolation Next, we interpolate vertically between the two horizontally interpolated values:

$$I(x, y) = I(x, y_0) \cdot (y_1 - y) + I(x, y_1) \cdot (y - y_0) \quad (7)$$

Similarly, defining the fractional vertical distance:

$$\Delta y = y - y_0 \in [0, 1] \quad (8)$$

We obtain:

$$I(x, y) = I(x, y_0) \cdot (1 - \Delta y) + I(x, y_1) \cdot \Delta y \quad (9)$$

Combined Formula Substituting equations (5) and (6) into equation (9), we get the final bilinear interpolation formula:

$$\begin{aligned} I(x, y) = & I(x_0, y_0) \cdot (1 - \Delta x) \cdot (1 - \Delta y) \\ & + I(x_1, y_0) \cdot \Delta x \cdot (1 - \Delta y) \\ & + I(x_0, y_1) \cdot (1 - \Delta x) \cdot \Delta y \\ & + I(x_1, y_1) \cdot \Delta x \cdot \Delta y \end{aligned} \quad (10)$$

This formula represents a weighted average where each corner pixel contributes proportionally to its inverse distance from the interpolation point. The weights satisfy the constraint that they sum to unity:

$$(1 - \Delta x)(1 - \Delta y) + \Delta x(1 - \Delta y) + (1 - \Delta x)\Delta y + \Delta x\Delta y = 1 \quad (11)$$

Coordinate Mapping: To resize an image from dimensions $(H_{\text{in}}, W_{\text{in}})$ to $(H_{\text{out}}, W_{\text{out}})$, each output pixel at position (i, j) is mapped back to input coordinates:

$$x = j \cdot \frac{W_{\text{in}} - 1}{W_{\text{out}} - 1}, \quad y = i \cdot \frac{H_{\text{in}} - 1}{H_{\text{out}} - 1} \quad (12)$$

This mapping ensures that corner pixels align exactly between input and output images.

2.2.2 Bicubic Interpolation

Motivation: While bilinear interpolation provides smooth results, it only considers the function values at four corners, leading to discontinuities in the gradient at pixel boundaries. Bicubic interpolation addresses this by using a two-dimensional cubic polynomial that matches not only the function values but also the first derivatives (f_x, f_y) and the cross-derivative (f_{xy}) at the four corners of each unit square. This ensures continuity of the function and its first derivatives, resulting in smoother interpolated images with better edge preservation and more accurate gradient representation.

Mathematical Derivation:

Bicubic interpolation constructs a two-dimensional cubic polynomial over the unit square $[0, 1] \times [0, 1]$. The general form of the bicubic polynomial is:

$$p(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j \quad (13)$$

This polynomial has 16 unknown coefficients $\{a_{ij}\}$, which are determined by imposing 16 boundary conditions at the four corners of the unit square.

Boundary Conditions At each of the four corners $(0, 0)$, $(1, 0)$, $(0, 1)$, and $(1, 1)$, we specify:

- Function value: $f(x, y)$
- Partial derivative in x: $f_x(x, y) = \frac{\partial f}{\partial x}$
- Partial derivative in y: $f_y(x, y) = \frac{\partial f}{\partial y}$
- Cross derivative: $f_{xy}(x, y) = \frac{\partial^2 f}{\partial x \partial y}$

These 4 conditions at 4 corners give us $4 \times 4 = 16$ constraints, which uniquely determine the 16 coefficients.

Matrix Formulation The bicubic polynomial can be expressed in matrix form as:

$$p(x, y) = \mathbf{X} \cdot \boldsymbol{\alpha} \cdot \mathbf{Y}^T \quad (14)$$

where:

- $\mathbf{X} = [1, x, x^2, x^3]$ is the x-coordinate vector
- $\mathbf{Y} = [1, y, y^2, y^3]$ is the y-coordinate vector
- $\boldsymbol{\alpha}$ is the 4×4 coefficient matrix

The coefficient matrix $\boldsymbol{\alpha}$ is computed from the boundary conditions using:

$$\boldsymbol{\alpha} = \mathbf{A} \cdot \mathbf{F} \cdot \mathbf{A}^T \quad (15)$$

where $\mathbf{F}$ is the 4×4 matrix containing the boundary condition values:

$$\mathbf{F} = \begin{bmatrix} f(0, 0) & f(0, 1) & f_y(0, 0) & f_y(0, 1) \\ f(1, 0) & f(1, 1) & f_y(1, 0) & f_y(1, 1) \\ f_x(0, 0) & f_x(0, 1) & f_{xy}(0, 0) & f_{xy}(0, 1) \\ f_x(1, 0) & f_x(1, 1) & f_{xy}(1, 0) & f_{xy}(1, 1) \end{bmatrix} \quad (16)$$

and $\mathbf{A}$ is the constant transformation matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -3 & 3 & -2 & -1 \\ 2 & -2 & 1 & 1 \end{bmatrix} \quad (17)$$

This matrix $\mathbf{A}$ is derived from the system of linear equations that enforce the 16 boundary conditions on the polynomial coefficients. For the detailed derivation, please see C or https://en.wikipedia.org/wiki/Bicubic_interpolation.

Derivative Estimation For discrete image data, the derivatives at each pixel are estimated using centered finite differences from the 4×4 neighborhood:

Partial derivatives in x:

$$f_x(x, y) = \frac{I(x+1, y) - I(x-1, y)}{2} \quad (18)$$

Partial derivatives in y:

$$f_y(x, y) = \frac{I(x, y+1) - I(x, y-1)}{2} \quad (19)$$

Cross derivatives:

$$f_{xy}(x, y) = \frac{I(x+1, y+1) - I(x+1, y-1) - I(x-1, y+1) + I(x-1, y-1)}{4} \quad (20)$$

These finite difference approximations require a 4×4 pixel neighborhood centered around the unit square being interpolated.

Algorithm Summary For each output pixel at fractional position $(\Delta x, \Delta y) \in [0, 1] \times [0, 1]$ within a unit square:

1. Extract the 4×4 neighborhood from the input image
2. Compute the function values at the 4 corners
3. Estimate derivatives f_x , f_y , and f_{xy} at the 4 corners using finite differences
4. Construct the boundary condition matrix $\mathbf{F}$
5. Calculate coefficient matrix: $\boldsymbol{\alpha} = \mathbf{A} \cdot \mathbf{F} \cdot \mathbf{A}^T$
6. Evaluate the polynomial: $p(\Delta x, \Delta y) = [1, \Delta x, \Delta x^2, \Delta x^3] \cdot \boldsymbol{\alpha} \cdot [1, \Delta y, \Delta y^2, \Delta y^3]^T$

Boundary Handling: To handle pixels near image borders where the 4×4 neighborhood extends beyond the image, we use edge replication (padding) to extend the image by 2 pixels on all sides before interpolation.

2.3 Performance Evaluation

Performance Measure: The quality of the interpolation methods was assessed using two standard full-reference metrics: the **Normalized Mean Square Error (NMSE)** and the **Structural Similarity Index Measure (SSIM)**.

- **NMSE** is a classical error metric that quantifies the average squared intensity difference between the interpolated image and the ground truth image, normalized by the reference power. A **lower NMSE** indicates better fidelity to the ground truth.

$$\text{NMSE} = \frac{\sum (I_{\text{interpolated}} - I_{\text{reference}})^2}{\sum (I_{\text{reference}})^2}$$

- **SSIM** is a perceptual metric designed to better correlate with the Human Visual System introduced in Zhou Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," in *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, April 2004, Zhou Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," in *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, April 2004. It evaluates image quality based on local structure, luminance, and contrast. A **higher SSIM** (closer to 1) indicates greater perceptual similarity.

Figure 1 illustrate the performance of bilinear interpolation with original downsampled images at the top row, interpolated images at the middle and error maps (absolute value of the difference between interpolated and the original image) at the bottom row. The error maps highlight the regions of maximum reconstruction error (darker areas indicate lower error). Figure 2 does the same for bicubic interpolation. The plots in Fig. 3 present the quantitative performance of

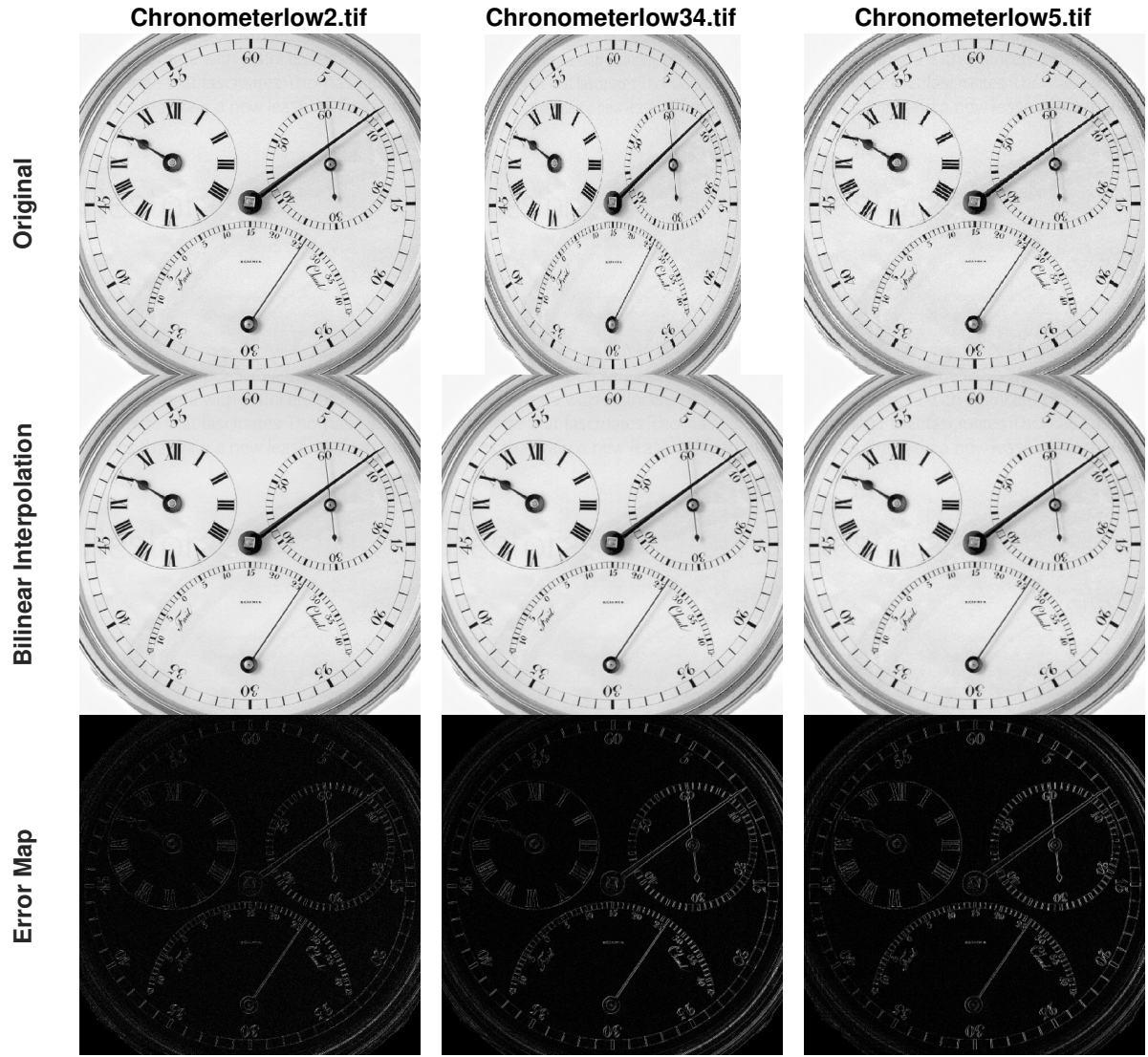


Figure 1: Bilinear Interpolation Results

bilinear and bicubic interpolation for the three downscaling factors using the calculated NMSE and SSIM values.

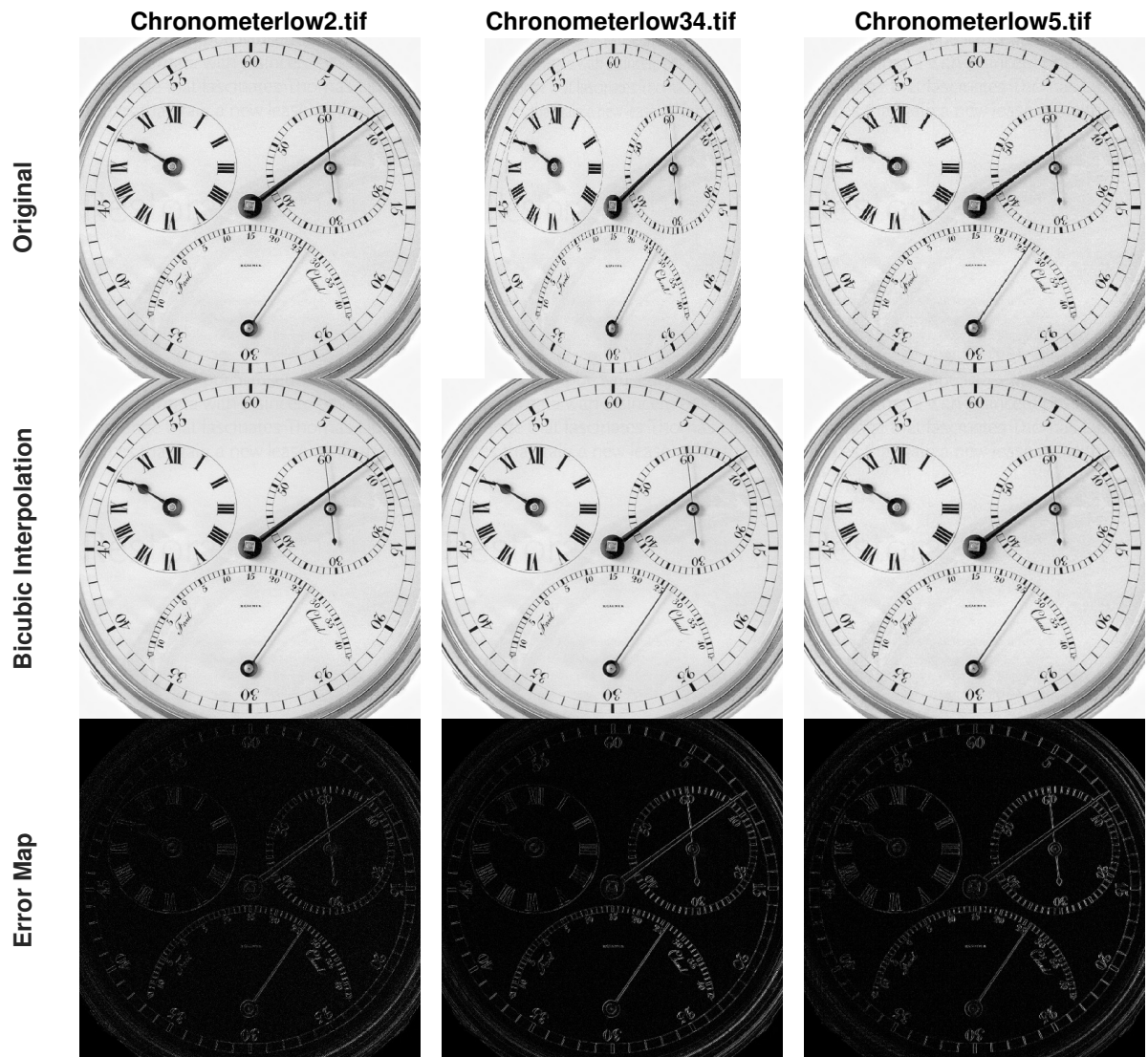


Figure 2: Bicubic Interpolation Results

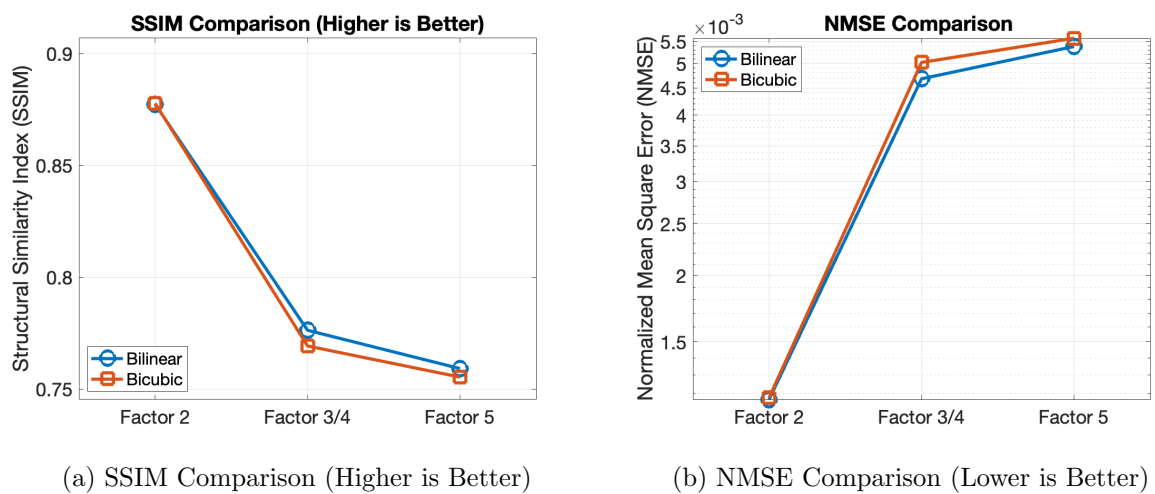


Figure 3: SSIM and NMSE Comparison

Discussion of Performance

Across both algorithms, an increase in the downscaling factor (from Factor 2 to Factor 5) consistently resulted in a sharp decrease in SSIM and a pronounced increase in NMSE. This trend confirms the expected degradation in reconstruction quality as the available source data for interpolation is reduced.

For the minimal degradation scenario (Factor 2), however, Bilinear interpolation exhibits marginally superior performance, with a higher SSIM and a lower NMSE compared to Bicubic interpolation. While Bicubic utilizes a 4×4 neighborhood (which inherently captures more information than bilinear's 2×2 neighborhood) to enforce continuity of first derivatives, which typically yields a smoother result, this complex reconstruction can introduce overshoot or ringing artifacts. These artifacts lead to greater absolute pixel error (higher NMSE) and can diminish the structural fidelity (SSIM) when compared to the simpler output of Bilinear interpolation. (See [this link](#) for more detail.)

A Scaling Code

```
1 function A_normalized = scale_image_to_01(A)
2     % Scale image matrix A to range [0, 1]
3     %
4     % Input:
5     %   A - input image matrix
6     % Output:
7     %   A_normalized - scaled image with values in [0, 1]
8
9     % Find minimum and maximum values in the image
10    min_val = min(A(:));
11    max_val = max(A(:));
12
13    % Handle edge case: constant image (avoid division by zero)
14    if max_val == min_val
15        A_normalized = zeros(size(A));
16        return;
17    end
18
19    % Apply normalization formula: (A - min) / (max - min)
20    A_normalized = (A - min_val) / (max_val - min_val);
21 end
```

Listing 1:]MATLAB implementation of image scaling to [0,1]

B Interpolation Code

B.1 Bilinear Interpolation

```
1
2
3 function output_image = bilinear_interpolation(input_image, target_size)
4     % Parameters:
5     % -----
6     % input_image : matrix / Input grayscale image as 2D array (height, width)
7     % target_size : vector [target_height, target_width]
8     %
9     % Returns:
10    % -----
11    % output_image : matrix / Interpolated image with target_size dimensions
12
13    [input_height, input_width] = size(input_image);
14    target_height = target_size(1);
15    target_width = target_size(2);
16
17    % Initialize output image
18    output_image = zeros(target_height, target_width, class(input_image));
19
20    % Compute scaling factors
21    % Subtract 1 to map edge pixels correctly
22    if target_height > 1
23        scale_y = (input_height - 1) / (target_height - 1);
24    else
25        scale_y = 0;
26    end
27
28    if target_width > 1
29        scale_x = (input_width - 1) / (target_width - 1);
30    else
31        scale_x = 0;
```

```

32     end
33
34     % Iterate through each pixel in output image
35     for i = 1:target_height
36         for j = 1:target_width
37             % Map output coordinates to input coordinates
38             % Note: MATLAB uses 1-based indexing, so adjust accordingly
39             y = (i - 1) * scale_y + 1;
40             x = (j - 1) * scale_x + 1;
41
42             % Find the four surrounding pixels
43             x0 = floor(x);
44             x1 = min(x0 + 1, input_width);
45             y0 = floor(y);
46             y1 = min(y0 + 1, input_height);
47
48             % Calculate fractional distances (delta_x, delta_y)
49             dx = x - x0;
50             dy = y - y0;
51
52             % Get the four corner pixel intensities
53             I00 = input_image(y0, x0); % Top-left
54             I10 = input_image(y0, x1); % Top-right
55             I01 = input_image(y1, x0); % Bottom-left
56             I11 = input_image(y1, x1); % Bottom-right
57
58             % Apply bilinear interpolation formula
59             output_image(i, j) = ...
60                 I00 * (1 - dx) * (1 - dy) + ...
61                 I10 * dx * (1 - dy) + ...
62                 I01 * (1 - dx) * dy + ...
63                 I11 * dx * dy;
64         end
65     end
66
67 end

```

Listing 2: MATLAB implementation of bilinear interpolation

B.2 Bi-cubic Interpolation

```

1 function output_image = bicubic_interpolation(input_image, target_size)
2 % Parameters:
3 % -----
4 % input_image : numeric matrix / Grayscale image as 2D array (height, width).
5 % target_size : 1x2 vector / [target_height, target_width].
6 %
7 % Returns:
8 % -----
9 % output_image : numeric matrix / Interpolated image.
10
11 % Get input and target dimensions
12 [input_height, input_width] = size(input_image);
13 target_height = target_size(1);
14 target_width = target_size(2);
15
16 % Store the original data type to cast the output at the end
17 original_class = class(input_image);
18
19 % Convert input to double for calculation precision
20 input_image = im2double(input_image);
21

```

```

22 % Initialize the output image as a double matrix
23 output_image = zeros(target_height, target_width);
24
25 % Compute scaling factors
26 % The (dim - 1) logic correctly maps the coordinate ranges
27 if target_height > 1
28     scale_y = (input_height - 1) / (target_height - 1);
29 else
30     scale_y = 0;
31 end
32
33 if target_width > 1
34     scale_x = (input_width - 1) / (target_width - 1);
35 else
36     scale_x = 0;
37 end
38
39 % For bicubic interpolation, we need a 4x4 neighborhood. Padding the image
40 % is the cleanest way to handle pixels near the borders.
41 % 'replicate' is MATLAB's equivalent of numpy's 'edge' mode.
42 padded_image = padarray(input_image, [2 2], 'replicate');
43
44 % Pre-calculate the constant bicubic interpolation matrix 'A'
45 A = [ 1  0  0  0;
46       0  0  1  0;
47      -3  3 -2 -1;
48       2 -2  1  1];
49
50 % Iterate through each pixel in the output image (using 1-based indexing)
51 for i = 1:target_height
52     for j = 1:target_width
53         % Map output coordinates (i, j) to 0-indexed input coordinates (y,
54         x)
55         y = (i - 1) * scale_y;
56         x = (j - 1) * scale_x;
57
58         % Account for the 2-pixel padding and convert to 1-based index
59         y_pad = y + 2 + 1;
60         x_pad = x + 2 + 1;
61
62         % Get integer and fractional parts
63         y0 = floor(y_pad);
64         x0 = floor(x_pad);
65         dy = y_pad - y0;
66         dx = x_pad - x0;
67
68         % Extract the 4x4 neighborhood patch (P) from the padded image.
69         % The four central points are centered around (x0, y0).
70         P = padded_image(y0 - 1 : y0 + 2, x0 - 1 : x0 + 2);
71
72         % --- Calculate derivatives for the 4 central points ---
73         % f(x,y) values (pixel intensities at the 4 corners of the unit
74         square)
75         f00 = P(2, 2); f10 = P(2, 3); f01 = P(3, 2); f11 = P(3, 3);
76
77         % fx (derivatives in x-direction)
78         fx00 = (P(2, 3) - P(2, 1)) / 2;
79         fx10 = (P(2, 4) - P(2, 2)) / 2;
80         fx01 = (P(3, 3) - P(3, 1)) / 2;
81         fx11 = (P(3, 4) - P(3, 2)) / 2;
82
83         % fy (derivatives in y-direction)
84         fy00 = (P(3, 2) - P(1, 2)) / 2;

```

```

83     fy10 = (P(3, 3) - P(1, 3)) / 2;
84     fy01 = (P(4, 2) - P(2, 2)) / 2;
85     fy11 = (P(4, 3) - P(2, 3)) / 2;
86
87     % fxy (cross derivatives)
88     fxy00 = (P(3, 3) - P(3, 1) - P(1, 3) + P(1, 1)) / 4;
89     fxy10 = (P(3, 4) - P(3, 2) - P(1, 4) + P(1, 2)) / 4;
90     fxy01 = (P(4, 3) - P(4, 1) - P(2, 3) + P(2, 1)) / 4;
91     fxy11 = (P(4, 4) - P(4, 2) - P(2, 4) + P(2, 2)) / 4;
92
93     % Construct the boundary condition matrix F
94     F = [f00,  f01,  fy00,  fy01;
95          f10,  f11,  fy10,  fy11;
96          fx00, fx01, fxy00, fxy01;
97          fx10, fx11, fxy10, fxy11];
98
99     % Calculate the coefficient matrix: alpha = A * F * A_transpose
100    alpha = A * F * A';
101
102    % Create the x and y vectors
103    X = [1, dx, dx^2, dx^3];
104    Y = [1, dy, dy^2, dy^3];
105
106    % The interpolated value is X * alpha * Y_transpose
107    output_image(i, j) = X * alpha * Y';
108  end
109 end
110
111 % Convert the final image back to the original data type (e.g., uint8)
112 % The conversion function (e.g., im2uint8) also clips values to the valid
113 % range.
114 if strcmp(original_class, 'uint8')
115     output_image = im2uint8(output_image);
116 elseif strcmp(original_class, 'uint16')
117     output_image = im2uint16(output_image);
118 end
119 % Add other types if needed, or just let it return double.
120 end

```

Listing 3: Bi-cubic interpolation implementation

C Derivation of A matrix

The bicubic polynomial is expressed as:

$$p(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j. \quad (21)$$

The interpolation problem consists of determining the 16 coefficients a_{ij} . Matching $p(x, y)$ with the function values yields four equations:

$$f(0, 0) = p(0, 0) = a_{00}, \quad (22)$$

$$f(1, 0) = p(1, 0) = a_{00} + a_{10} + a_{20} + a_{30}, \quad (23)$$

$$f(0, 1) = p(0, 1) = a_{00} + a_{01} + a_{02} + a_{03}, \quad (24)$$

$$f(1, 1) = p(1, 1) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij}. \quad (25)$$

Likewise, eight equations for the derivatives in the x and y directions:

$$f_x(0,0) = p_x(0,0) = a_{10}, \quad (26)$$

$$f_x(1,0) = p_x(1,0) = a_{10} + 2a_{20} + 3a_{30}, \quad (27)$$

$$f_x(0,1) = p_x(0,1) = a_{10} + a_{11} + a_{12} + a_{13}, \quad (28)$$

$$f_x(1,1) = p_x(1,1) = \sum_{i=1}^3 \sum_{j=0}^3 i a_{ij}, \quad (29)$$

$$f_y(0,0) = p_y(0,0) = a_{01}, \quad (30)$$

$$f_y(1,0) = p_y(1,0) = a_{01} + a_{11} + a_{21} + a_{31}, \quad (31)$$

$$f_y(0,1) = p_y(0,1) = a_{01} + 2a_{02} + 3a_{03}, \quad (32)$$

$$f_y(1,1) = p_y(1,1) = \sum_{i=0}^3 \sum_{j=1}^3 j a_{ij}. \quad (33)$$

And four equations for the xy mixed partial derivatives:

$$f_{xy}(0,0) = p_{xy}(0,0) = a_{11}, \quad (34)$$

$$f_{xy}(1,0) = p_{xy}(1,0) = a_{11} + 2a_{21} + 3a_{31}, \quad (35)$$

$$f_{xy}(0,1) = p_{xy}(0,1) = a_{11} + 2a_{12} + 3a_{13}, \quad (36)$$

$$f_{xy}(1,1) = p_{xy}(1,1) = \sum_{i=1}^3 \sum_{j=1}^3 ij a_{ij}. \quad (37)$$

The expressions above have used the following identities:

$$p_x(x,y) = \sum_{i=1}^3 \sum_{j=0}^3 a_{ij} i x^{i-1} y^j, \quad (38)$$

$$p_y(x,y) = \sum_{i=0}^3 \sum_{j=1}^3 a_{ij} j x^i y^{j-1}. \quad (39)$$

$$p_{xy}(x,y) = \sum_{i=1}^3 \sum_{j=1}^3 a_{ij} i x^{i-1} j y^{j-1} \quad (40)$$

There can be a concise matrix form for the 16 coefficients:

$$\begin{bmatrix} f(0,0) & f(0,1) & f_y(0,0) & f_y(0,1) \\ f(1,0) & f(1,1) & f_y(1,0) & f_y(1,1) \\ f_x(0,0) & f_x(0,1) & f_{xy}(0,0) & f_{xy}(0,1) \\ f_x(1,0) & f_x(1,1) & f_{xy}(1,0) & f_{xy}(1,1) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{bmatrix}. \quad (41)$$

Alternatively, a more compact matrix formulation for the coefficients a_{ij} is:

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -3 & 3 & -2 & -1 \\ 2 & -2 & 1 & 1 \end{bmatrix} \begin{bmatrix} f(0,0) & f(0,1) & f_y(0,0) & f_y(0,1) \\ f(1,0) & f(1,1) & f_y(1,0) & f_y(1,1) \\ f_x(0,0) & f_x(0,1) & f_{xy}(0,0) & f_{xy}(0,1) \\ f_x(1,0) & f_x(1,1) & f_{xy}(1,0) & f_{xy}(1,1) \end{bmatrix} \begin{bmatrix} 1 & 0 & -3 & 2 \\ 0 & 0 & 3 & -2 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & -1 & 1 \end{bmatrix}. \quad (42)$$

Finally, the interpolating surface can be evaluated as:

$$p(x, y) = \begin{bmatrix} 1 & x & x^2 & x^3 \end{bmatrix} \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} 1 \\ y \\ y^2 \\ y^3 \end{bmatrix}. \quad (43)$$